

Assignment-16 (Lab-16.5)

Course : AI Assisted Coding

Topic: Database Design and Queries

Student Details

Name :Batti Chandan Singh

Hall Ticket :2303A51515

Batch :22

Task-01

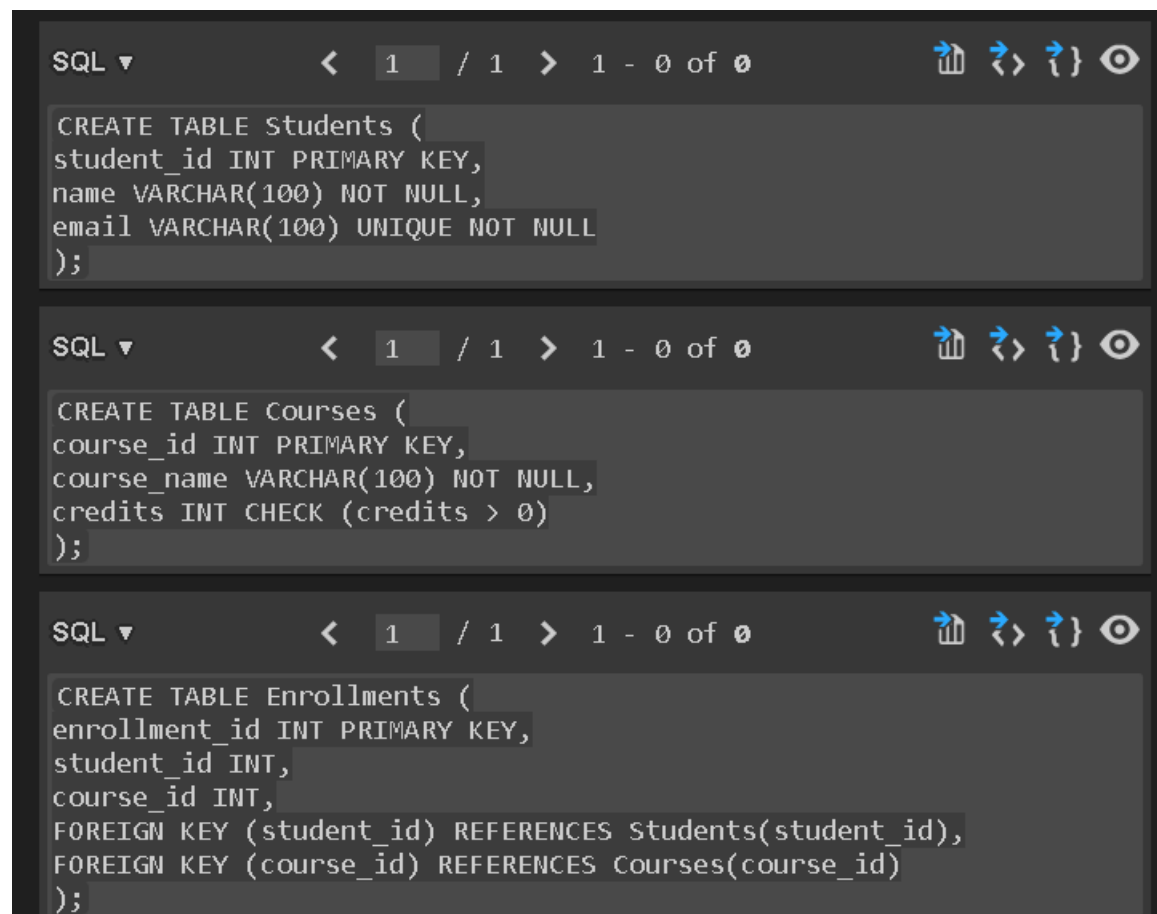
Final Optimal Prompt

Generate SQL CREATE TABLE statements for a Student Management System with Students, Courses, and Enrollments tables Include primary keys, foreign keys, proper data types, and ensure normalization (1NF, 2NF, 3NF). Handle constraints like NOT NULL, UNIQUE, and valid values.

Code Screenshot

```
Assignments python file > ForSQL > Task1.sql
1  CREATE TABLE Students (
2      student_id INT PRIMARY KEY,
3      name VARCHAR(100) NOT NULL,
4      email VARCHAR(100) UNIQUE NOT NULL
5  );
6
7  CREATE TABLE Courses (
8      course_id INT PRIMARY KEY,
9      course_name VARCHAR(100) NOT NULL,
10     credits INT CHECK (credits > 0)
11 );
12
13 CREATE TABLE Enrollments (
14     enrollment_id INT PRIMARY KEY,
15     student_id INT,
16     course_id INT,
17     FOREIGN KEY (student_id) REFERENCES Students(student_id)
18     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
19 );
```

Output Screenshot

The screenshot shows three separate SQL query editors, each with a dark background and light text. Each editor has a title bar with 'SQL' and a dropdown arrow, and a status bar with navigation icons and text like '< 1 / 1 > 1 - 0 of 0'. The first editor contains the 'CREATE TABLE Students' query. The second editor contains the 'CREATE TABLE Courses' query. The third editor contains the 'CREATE TABLE Enrollments' query, which includes foreign key constraints.

```
SQL ▾ < 1 / 1 > 1 - 0 of 0
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL
);

SQL ▾ < 1 / 1 > 1 - 0 of 0
CREATE TABLE Courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(100) NOT NULL,
  credits INT CHECK (credits > 0)
);

SQL ▾ < 1 / 1 > 1 - 0 of 0
CREATE TABLE Enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT,
  course_id INT,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task focuses on designing a relational database schema using normalization principles. Three tables are created: Students, Courses, and Enrollments. Each table has a primary key to uniquely identify records. The Enrollments table uses foreign keys to connect students and courses, ensuring referential integrity. Constraints such as NOT NULL, UNIQUE, and CHECK improve data validation and prevent incorrect entries. The schema follows normalization rules: 1NF ensures atomic values, 2NF removes partial dependencies, and 3NF eliminates transitive dependencies. This design reduces redundancy and improves efficiency. It is scalable and suitable for real-world academic systems managing multiple student enrollments.

Task-02

Final Optimal Prompt

Generate SQL INSERT statements to populate Students, Courses, and Enrollments tables with at least 3 students, 4 courses, and 5 enrollments.

Ensure valid data and allow verification using SELECT queries.

Code Screenshot

```
--Task-02--
--Generate SQL INSERT statements to populate Students, Courses, and Enrollment
--Ensure valid data and allow verification using SELECT queries.
-- INSERT INTO Students VALUES
-- (1, 'Arjun', 'arjun@gmail.com'),
-- (2, 'Meera', 'meera@gmail.com'),
-- (3, 'Rahul', 'rahul@gmail.com');

-- INSERT INTO Courses VALUES
-- (101, 'Database Systems', 4),
-- (102, 'Data Structures', 3),
-- (103, 'Operating Systems', 4),
-- (104, 'AI Fundamentals', 3);

-- INSERT INTO Enrollments VALUES
-- (1, 1, 101),
-- (2, 1, 102),
-- (3, 2, 103),
-- (4, 3, 101),
-- (5, 3, 104);
SELECT * FROM Students;
```

Output Screenshot

student_id	name	email
1	Arjun	arjun@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task involves inserting sample data into the database tables. Three student records, four course records, and five enrollment entries are added using INSERT statements. Each record follows the schema constraints to maintain data integrity. The Enrollments table correctly links students and courses through foreign keys. SELECT queries are used to verify that the data is inserted successfully. This step ensures that the database is functioning correctly and relationships are properly maintained. It simulates real-world scenarios where multiple students enroll in different courses. Proper data insertion is essential before performing operations like updates, deletions, or analysis queries.

Task-03

Final Optimal Prompt

Generate SQL queries to perform CRUD operations: retrieve all courses, update a student's email, delete an enrollment record, and verify changes using SELECT queries.

Ensure correctness and data consistency.

Code Screenshot

```
--Task-03--
-- Generate SQL queries to perform CRUD operations
-- Ensure correctness and data consistency.
SELECT * FROM Courses;

UPDATE Students
SET email = 'anjun_new@gmail.com'
WHERE student_id = 1;

DELETE FROM Enrollments
WHERE enrollment_id = 5;

SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output Screenshot

SQL ▾ < 1 / 1 > 1 - 0 of 0

UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;

SQL ▾ < 1 / 1 > 1 - 0 of 0

DELETE FROM Enrollments
WHERE enrollment_id = 5;

SQL ▾ < 1 / 1 > 1 - 3 of 3

student_id	name	email
1	Anjun	arjun_new@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

SQL ▾ < 1 / 1 > 1 - 4 of 4

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task demonstrates CRUD operations in SQL. The SELECT query retrieves all course records from the Courses table. The UPDATE statement modifies a student's email address, showing how data can be updated dynamically. The DELETE query removes a specific enrollment record, demonstrating data removal. After performing update and delete operations, SELECT queries are used to verify the changes. This ensures that only intended records are modified or deleted. These operations are fundamental for managing databases efficiently. They help maintain accurate and up-to-date information while preserving data integrity across related tables in a relational database system.

Task-04

Final Optimal Prompt

Generate SQL aggregate queries to count the number of students per course and display courses with more than one student enrolled.

Use JOIN, GROUP BY, and HAVING clauses.

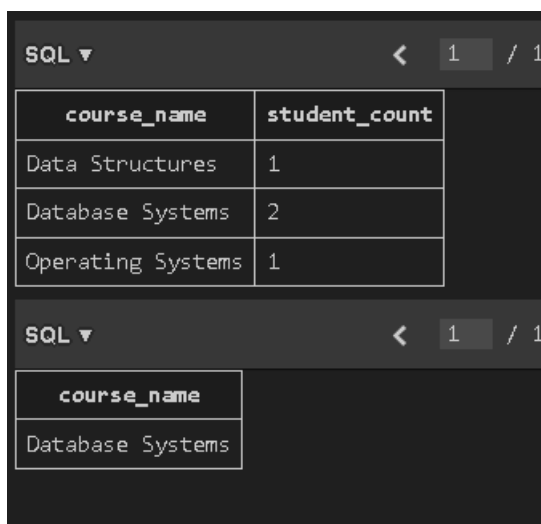
Code Screenshot

```
-- --Task-04--
-- Generate SQL aggregate queries to count the number of students per course.
-- Use JOIN, GROUP BY, and HAVING clauses.

SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;

SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Output Screenshot



course_name	student_count
Data Structures	1
Database Systems	2
Operating Systems	1

course_name
Database Systems

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task focuses on data analysis using SQL aggregate functions. The COUNT function is used to calculate how many students are enrolled in each course. The GROUP BY clause groups the data based on course names. The HAVING clause filters results to display only courses with more than one student enrolled. JOIN operations combine data from Courses and Enrollments tables to enable meaningful analysis. These queries help understand course popularity and student distribution. Aggregate queries are widely used in reporting and decision-making systems. This task shows how SQL can be used not only for storage but also for extracting valuable insights from data.

Task-05

Final Optimal Prompt

Compare a subquery-based SQL query with an optimized JOIN-based query to fetch students enrolled in a course.

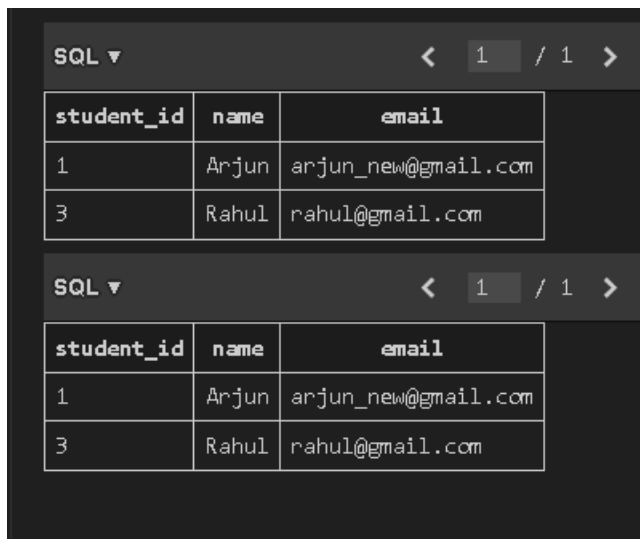
Ensure both return the same result and explain performance improvement.

Code Screenshot

```
-- --Task-05--
-- Compare a subquery-based SQL query with an optimized JOIN-based query
-- Ensure both return the same result and explain performance improvement
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
    SELECT student_id FROM Enrollments WHERE course_id = 101
);

-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Output Screenshot



The screenshot displays two identical SQL query result sets. Each result set is presented in a table with three columns: student_id, name, and email. The first result set shows two rows of data: student_id 1 (Anjun, arjun_new@gmail.com) and student_id 3 (Rahul, rahul@gmail.com). The second result set shows the same two rows of data.

student_id	name	email
1	Anjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com

student_id	name	email
1	Anjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task demonstrates query optimization techniques in SQL. The original query uses a subquery to find students enrolled in a specific course. The optimized query replaces the subquery with a JOIN operation. JOINS are generally more efficient because they directly combine tables instead of executing nested queries. Both queries produce the same result, ensuring correctness. However, the JOIN-based query performs better, especially with large

datasets. This improvement reduces execution time and enhances performance. Query optimization is important in real-world applications where databases handle large volumes of data. AI tools help identify and suggest efficient query structures automatically.